Machine Translated by Google

MODUŁ 1: Wprowadzenie do kontroli wersji

Cel: Zrozumieć czym jest Git i dlaczego jest ważny.

Czym jest kontrola wersji?

Problemy rozwiązywane przez Git

Git kontra inne systemy kontroli wersji

Czym jest Git i kto go stworzył?

Przykłady zastosowań w świecie rzeczywistym (praca zespołowa, projekty osobiste)

Instalacja Git (Windows, macOS, Linux)

Pierwszy kontakt z terminalem

MODUŁ 2: Początkowa konfiguracja Git

Cel: Przygotowanie Gita do użycia.

Konfiguracja użytkownika (nazwa użytkownika i adres e-mail użytkownika)

Poziomy konfiguracji (systemowy, globalny, lokalny)

polecenie git config

Sprawdź konfigurację

Pomoc Git

MODUŁ 3: Repozytoria i podstawowy przepływ pracy

Cel: Poznanie podstawowego przepływu pracy z Gitem.

Czym jest repozytorium?

Utwórz repozytorium (git init)

Stany plików w Git

Katalog roboczy

Obszar przygotowawczy

Magazyn

Machine Translated by Google

status git

dodaj git

zatwierdzenie git

Najlepsze praktyki dotyczące komunikatów zatwierdzających

Historia zmian (git log)

MODUŁ 4: Zarządzanie plikami i zmianami

Cel: Kontrolowanie i przeglądanie zmian w projekcie.

Wyświetl różnice (git diff)

Modyfikuj pliki i śledź zmiany

Usuń pliki (git rm)

Zmień nazwy plików (git mv)

Ignoruj pliki (.gitignore)

MODUŁ 5: Gałęzie

Cel: Praca z równoległymi wersjami projektu.

Czym jest oddział?

Oddział główny/główny

Utwórz gałęzie (git branch)

Przełączanie gałęzi (git checkout, git switch)

Scalenie gałęzi (git merge)

Konflikty związane z fuzjami

Rozwiązywanie konfliktów

Machine Translated by Google

MODUŁ 6: Zdalne repozytoria (GitHub / GitLab / Bitbucket)

Cel: Praca z repozytoriami w chmurze.

Czym jest zdalne repozytorium?

Załóż konto na GitHub (lub innej platformie)

Klonowanie repozytoriów (git clone)

Połącz lokalne repozytorium ze zdalnym repozytorium

git push

git pull

pobierz git

Podstawowy przepływ pracy w trybie współpracy

MODUŁ 7: Praca zespołowa

Cel: Wykorzystanie Gita w zespołach.

Widelce

Żądania ściągnięcia / żądania scalenia

Przegląd kodu

Dobre praktyki w zespołach

Strategie rozgałęzień (Git Flow – wprowadzenie)

MODUŁ 8: Cofanie zmian

Cel: Korygować błędy bez strachu

Cofnij niepotwierdzone zmiany

git checkout --

przywróć git

git reset (miękki, mieszany, twardy)

przywróć git

Machine Translated by Google

Kiedy używać każdego z nich

MODUŁ 9: Zaawansowany Git (opcjonalnie)

Cel: głębsze poznanie Gita.

git stash

Rebase (git rebase)

Wybiórczo

Tagi i wersje

Git Hooks (Wprowadzenie)

Alias w Git

MODUŁ 10: Dobre praktyki i projekt końcowy

Cel: Utrwalenie nabytej wiedzy.

Konwencje commitów

Zalecany przepływ pracy

Typowe błędy w Git

Machine Translated by Google

GIT – MODUŁ 1

Cel

Zrozumienie, czym jest kontrola wersji i dlaczego Git jest niezbędnym narzędziem w Nowoczesny rozwój i jak zacząć go używać od podstaw na różnych systemach operacyjnych.

Czym jest kontrola wersji?

Kontrola wersji to system rejestrujący zmiany wprowadzane w zestawie plików na przestrzeni czasu, umożliwiający powrót do poprzednich wersji, porównywanie modyfikacji i współpracę z innymi użytkownikami.

W kontekście rozwoju oprogramowania stosuje się go głównie w celu zapisywania historii kodu źródłowego i ułatwiania pracy zespołowej.

Prosty przykład:

Pomyśl o dokumencie, który edytujesz codziennie. Bez kontroli wersji musiałbyś zapisywać kopie pod nazwami takimi jak project_v1, final_project, final_project_definitive.

Dzięki kontroli wersji każda zmiana jest rejestrowana, dzięki czemu możesz sprawdzić, kto, co i kiedy zmienił, oraz cofnąć zmiany, jeśli coś pójdzie nie tak.

Problemy rozwiązywane przez Git

Git oferuje rozwiązania typowych problemów związanych z pracą z kodem i plikami:

- Unikaj utraty pracy: Każda wersja jest przechowywana i można ją odzyskać. odzyskiwać.
- Umożliwia pracę zespołową: Kilka osób może edytować ten sam dokument projektu w tym samym czasie, bez nadpisywania swojej pracy.
- Ułatwia scalanie zmian: gdy dwóch programistów modyfikuje ten sam plik, Git pomaga bezpiecznie połączyć te zmiany.
- Ułatwia śledzenie: Można dowiedzieć się, jaka zmiana spowodowała błąd lub kiedy została wprowadzona nowa funkcjonalność.
- Wsparcie dla rozgałęzień: Rozgałęzienia umożliwiają rozwój nowych funkcje, testowanie pomysłów lub naprawianie błędów bez wpływu na główną wersję kodu.

Machine Translated by Google

Git kontra inne systemy kontroli wersji

Przed Gitem istniały narzędzia takie jak CVS (Concurrent Versions System) czy Subversion (SVN).
Narzędzia te wykorzystywały scentralizowane podejście, w którym wszystkie zmiany musiały przechodzić przez jeden serwer.

Git wprowadził model rozproszony, który oferuje szereg zalet:

Funkcja	Systemy scentralizowane (SVN, CVS)	Git
Model roboczy: serwer centralny		Każdy użytkownik posiada kompletną kopię repozytorium.
Dostęp offline	Ograniczone lub nieistniejące	Kompletny, umożliwia zatwierdzanie i eksplorację offline
Prędkość	Wolniej	Bardzo szybko
Łączenie gałęzi	Złożony	Wydajny i elastyczny
Bezpieczeństwo	Ryzyko w przypadku awarii serwera	Pełna kopia na każdym komputerze

Dzięki takiemu rozproszonemu modelowi Git stał się praktycznie standardem w nowoczesnej kontroli wersji.

Czym jest Git i kto go stworzył?

Git to rozproszony system kontroli wersji o otwartym kodzie źródłowym, zaprojektowany do szybkiego i wydajnego zarządzania projektami dowolnej wielkości.
Został stworzony w 2005 roku przez Linusa Torvaldsa, tego samego twórcę jądra Linux, w celu zarządzania rozwojem samego systemu operacyjnego Linux po problemach z poprzednimi narzędziami.

Główne cechy Gita:

- Dystrybuowane (każda kopia stanowi kompletne repozytorium).
- Szybki i lekki.
- Elastyczność: umożliwia różne przepływy pracy (indywidualne, zespołowe, zdalne, itp.).
- Zintegrowany z platformami takimi jak GitHub, GitLab lub Bitbucket w celu umożliwienia współpracy w trybie online.

Machine Translated by Google

Przykłady zastosowań w świecie rzeczywistym

Git jest używany w różnych scenariuszach:

- Praca zespołowa: Zespoły programistyczne współpracują ze sobą, nie nadpisując wzajemnie swojego kodu, korzystając gałęzie dla nowych funkcji lub poprawek.
- Projekty osobiste: Pojedynczy programista może prowadzić historię pełne śledzenie postępów i cofanie błędów.
- Firmy technologiczne: Stanowi podstawę ciągłej integracji (CI/CD), automatycznego testowania i wdrażania.
- Oprogramowanie typu Open Source: tysiące projektów publicznych korzysta z GitHub, GitLab, BitBucket i Git w celu koordynowania globalnych wkładów (takich jak React, WordPress czy TensorFlow).

Przykład:

Trzyosobowy zespół programistów tworzy gałąź dla nowej funkcjonalności, kolejny naprawia błędy w gałęzi konserwacyjnej, a trzeci przegląda kod na zdalnej platformie.

Git koordynuje wszystkie te przepływy bez konfliktów.

Instalacja Gita

Git jest dostępny dla wszystkich głównych systemów operacyjnych.

Okna:
Pobierz instalator ze strony git-scm.com, uruchom plik i postępuj zgodnie z instrukcjami. Instaluje się on wraz z Git Bash, terminalem kompatybilnym z systemem Unix.

macOS:
Można go zainstalować za pomocą Homebrew za pomocą polecenia:

```
instalacja git
```

lub pobierając oficjalny instalator z tej samej strony internetowej.

Linux (Debian/Ubuntu):

```
aktualizacja sudo apt
sudo apt zainstaluj git
```

Po instalacji możesz sprawdzić wersję za pomocą:

```
git --version
```

Machine Translated by Google

Pierwszy kontakt z terminalem

Git jest obsługiwany głównie z poziomu wiersza poleceń (choć istnieją interfejsy graficzne).

Oto kilka podstawowych poleceń na początek:

```
2 git config --global user.name "Twoje imię"
3 git config --global user.email "youremail@domain.com"
4 git init
5 status git
6 git add .
7 git commit -m "Pierwsze zatwierdzenie"
```

Te wstępne kroki skonfigurują Twoją tożsamość i umożliwią Ci utworzenie pierwszego lokalnego repozytorium.

Dzięki temu możesz rozpocząć poznawanie przepływu pracy Git w kolejnym module.

Machine Translated by Google

MODUŁ 10: Dobre praktyki i projekt końcowy

Cel

Utrwalaj wiedzę, stosując standardowe konwencje branżowe i uzupełniając je prawdziwy, praktyczny projekt.

Konwencje commitów

Konwencjonalne zatwierdzenia w wersji 1.0.0 (oficjalny standard):

```
<typ>[zakres opcjonalny]: <opis>

[treść opcjonalna]

[opcjonalne stopki]
```

Zalecane typy w branży

wyczyn	Nowa funkcja
naprawić	Naprawiono błąd
dokumenty	Tylko dokumentacja
styl	Zmiany formatu (bez logiki)
refaktoryzacja	Refaktoryzacja (bez nowej logiki)
perf	Poprawa wydajności
test	Dodaj lub popraw testy
zbudować	Zmiany w kompilacji lub zależnościach
ci	Zmiany w CI/CD
obowiązek	Zadania konserwacyjne
odwracać	Przywróć poprzednie zatwierdzenie

Przykłady

```
1 git commit -m „feat: zmodyfikowano nagłówek głównego szablonu”

2 git commit .m „naprawiono: rozwiązano problem z uwierzytelnianiem użytkownika ”

3 git commit -m „docs: Dodano dokumentację instalacji z
„Środowisko Docker”
```

Machine Translated by Google

Zalecany przepływ pracy

Indywidualny programista

```
1 git checkout main
2 git pull origin main
3 git checkout -b feature/feature-name
4 # Rozwijaj
5 git add .
6 git commit -m "feat: clear description"
7 git push origin feature/feature-name
8 # Utwórz żądanie ściągnięcia na GitHub
```

Programista zespołu

```
1 git checkout main
2 git pull origin main
3 git checkout develop
4 git pull origin develop
5 git checkout -b feature/feature-name
6 # Rozwijaj + zatwierdzaj
7 git rebase develop
8 git push origin feature/feature-name
9 # Pull Request develop     main
```

Machine Translated by Google

Typowe błędy w Git

Problem	Przyczyna	Rozwiązanie
nic do zobowiązania	Zapomniałeś o git add	git add .
gałąź jest do przodu o X zatwierdzeń	Masz lokalne zatwierdzenia	git push
zmiany zostaną nadpisane	Zmiany lokalne blokują ściąganie	git stash git pull git stash pop
odmowa połączenia niepowiązanych ze sobą historii	Repozytoria bez wspólnej historii	git pull --allow-unrelated-histories
odrzucone (nie można przewinąć do przodu)	Ktoś inny pchnął	git pull --rebase git push
oderwana GŁOWA	Sprawdź commit/hash	git checkout główny

Polecenia diagnostyczne

```
1 status git

2 git log --oneline -5

3 gałąź git -vv
```

Machine Translated by Google

MODUŁ 2: Początkowa konfiguracja Git

Cel

Zanim zaczniesz pracę, dowiedz się, jak poprawnie skonfigurować Gita. Ta wstępna konfiguracja definiuje tożsamość użytkownika, zakres jego ustawień oraz sposób uzyskiwania pomocy w środowisku Gita.

Konfiguracja użytkownika (nazwa użytkownika i adres e-mail użytkownika)

Za każdym razem, gdy zatwierdzasz zmiany, Git przypisuje tę czynność autorowi.

Dlatego pierwszym krokiem po zainstalowaniu Gita jest podanie nazwy i adresu e-mail, które będą używane do rejestrowania zmian.

Można je skonfigurować z poziomu terminala w następujący sposób:

```
1 git config --global user.name "Twoje imię"

2 git config --global user.email "youremail@domain.com"
```

Przykład:

```
1 git config --global user.name „Ana López”

2 git config --global user.email "ana.lopez@example.com"
```

Dane te pojawią się obok każdego potwierdzenia, które wykonasz, a także zostaną wyświetlone, jeśli udostępnisz swoją pracę na platformach typu GitHub lub GitLab.

Poziomy konfiguracji (systemowy, globalny, lokalny)

Git oferuje trzy poziomy konfiguracji, co zapewnia dużą elastyczność. Pojedynczy użytkownik może korzystać z różnych ustawień w zależności od kontekstu lub projektu.

Poziom	Lokalizacja pliku	Dotyczy...	Skonfiguruj za pomocą
System	/etc/gitconfig (Linux i macOS) C:\ProgramData\Git\config (Windows)	Do wszystkich użytkowników systemu	git config --system
Światowy	~/.gitconfig lub ~/.config/git/config (Linux i macOS) C:\Users\<użytkownik>\.gitconfig (Windows)	Tylko dla bieżącego użytkownika	git config --global
Lokalny	<katalog-repozytorium>/.git/config (wszystkie systemy)	Tylko do bieżącego repozytorium lub projektu	git config --local (domyślny)

Machine Translated by Google

Przykład praktyczny

W firmie możesz mieć skonfigurowaną pocztę korporacyjną na poziomie globalnym.

W przypadku projektu osobistego możesz użyć innego adresu e-mail w konfiguracji lokalnej.

Git stosuje następującą kolejność priorytetów: lokalny globalny systemowy

Innymi słowy, wartość lokalna nadpisuje wartość globalną, a wartość globalna nadpisuje wartość systemową.

polecenie git config

Polecenie git config służy do odczytu i zapisu wartości w plikach konfiguracyjnych Git.

Jego ogólna postać jest następująca:

git config [opcje] klucz wartość

Niektóre typowe zastosowania:

Ustaw informacje o użytkowniku:

```
1 git config --global user.name "Twoje imię"
2 git config --global user.email "youremail@domain.com"
```

Wyświetl wartość klucza:

```
1 git config nazwa_użytkownika
```

Wyświetl wszystkie aktywne ustawienia:

```
1 git config --list
```

Edytuj plik konfiguracji globalnej za pomocą edytora:

```
1 git config --global --edit
```

Polecenie to można wykorzystać zarówno do określania osobistych preferencji (kolor, edytor, scalanie itp.), jak i do dostosowywania technicznych zachowań Gita.

Sprawdź konfigurację

Aby mieć pewność, że ustawienia zostały zapisane prawidłowo, użyj:

Machine Translated by Google

```
1 git config –list
```

Wyświetli się pełna lista aktywnych ustawień.

Każdy wiersz wskazuje parę klucz=wartość, na przykład:

```
1 user.name=Ana López
2 użytkownik.email=ana.lopez@example.com
3 core.editor=vim
4 color.ui=auto
```

Możesz również sprawdzić konkretne ustawienia:

```
1 git config user.name
```

Wyświetli tylko aktualną wartość user.name.

Pomoc Git

Git zawiera bardzo przydatny system pomocy umożliwiający wyszukiwanie informacji o dowolnym poleceniu.

Trzy główne sposoby uzyskania pomocy:

Korzystając z polecenia help:

```
1     git help <polecenie>
# Przykład
5 git help commit
7 Z opcją --help:
9 git <polecenie> --help
11 Z opcją -h (wersja skrócona):
13 git <polecenie> -h
```

Pierwszy format otwiera pełną dokumentację w przeglądarce lub pagerze systemu.

Machine Translated by Google

Pozostałe dwa pokazują krótkie podsumowanie w terminalu.

Przykład praktyczny:

Jeśli nie pamiętasz, jak dodać pliki do obszaru roboczego, możesz wpisać:

```
1 git help dodaj
```

a Git wyświetli opis, opcje i przykłady polecenia git add.

Dzięki tej początkowej konfiguracji Git jest gotowy do użycia w dowolnym projekcie.

Twoja tożsamość jest już zarejestrowana, znasz rodzaje konfiguracji i wiesz, gdzie uzyskać pomoc, gdy jej potrzebujesz.

Machine Translated by Google

MODUŁ 3: Repozytoria i podstawowy przepływ pracy

Cel:

Zrozumieć, czym jest repozytorium w Git i opanować podstawowy przepływ pracy: od tworzenia repozytorium po śledzenie statusu plików.

Czym jest repozytorium?

Repozytorium to struktura, w której Git przechowuje wszystkie informacje dotyczące projektu: historię zmian, gałęzie, tagi i konkretną konfigurację danego projektu.

W praktyce jest to po prostu folder w systemie plików, nad którym kontrolę sprawuje Git.

Kiedy inicjujesz Git w folderze, tworzony jest ukryty podkatalog o nazwie .git.

W tym katalogu znajduje się cała wewnętrzna baza danych Git (zatwierdzenia, obiekty, odniesienia itd.).

Dopóki katalog .git istnieje, folder ten jest uważany za repozytorium Git.

Utwórz repozytorium (git init)

Aby utworzyć nowe repozytorium od podstaw, użyj następującego polecenia:

```
1 git init
```

Typowe kroki:

- Tworzysz folder dla swojego projektu (na przykład my-project).
- Dostęp do tego folderu uzyskuje się z terminala:
- mój projekt cd
- Zainicjuj repozytorium:

```
1 git init
```

Po wykonaniu tej czynności folder my-project stanie się repozytorium Git i wewnątrz niego znajdziesz folder .git (zwykle ukryty).

Od tej chwili wszystkie zmiany wprowadzone w plikach będą mogły być śledzone przez Git.

Możesz również zainicjować Git w folderze, w którym już zostały utworzone pliki.

Wystarczy przejść do tego folderu i uruchomić git init.

Machine Translated by Google

Stany plików w Git

Git zarządza plikami projektu poprzez różne stany, które określają, na jakim etapie przepływu pracy znajduje się każdy plik.

Główne stany to:

Nieśledzone: pliki, które istnieją w folderze, ale których Git jeszcze nie śledzi.

Śledzone: Pliki, o których Git już wie i które mogą być:

Bez modyfikacji: Nie wprowadzono żadnych zmian w porównaniu do ostatniego zatwierdzenia.

Zmodyfikowano: Zmieniono je, ale nie są jeszcze gotowe do zatwierdzenia.

Przygotowane: Zostały dodane do obszaru przygotowawczego i są gotowe do uwzględnienia w kolejnym zatwierdzeniu.

Status plików możesz sprawdzić w dowolnym momencie za pomocą:

```
status git
```

To polecenie informuje, które pliki nie są śledzone, które zostały zmodyfikowane i które są gotowe do następnego zatwierdzenia.

Katalog roboczy

Katalog roboczy to folder, w którym znajdują się pliki, które przeglądasz i edytujesz w swoim systemie.

Jest to „widzialna kopia" projektu, z którą pracujesz na co dzień.

Najważniejsze cechy:

- Tutaj możesz tworzyć, edytować, zmieniać nazwy i usuwać pliki.
- Zmiany wprowadzane w katalogu roboczym nie są jeszcze częścią Historia Git.
- Dopóki nie użyjesz poleceń git add i git commit, Git nie zatwierdzi tych zmian.

Przykład:

Otwierasz edytor i modyfikujesz plik index.php.

Ten plik będzie znajdował się w zmodyfikowanym stanie w Twoim katalogu roboczym, aż do momentu przygotowania go do zatwierdzenia.

Machine Translated by Google

Obszar przygotowawczy

Obszar przygotowawczy (nazywany również obszarem indeksowania lub obszarem przygotowawczym) to strefa przejściowa, w której wybierasz konkretne zmiany, które chcesz uwzględnić w kolejnym zatwierdzeniu.

Aby dodać zmodyfikowane lub nowe pliki do obszaru przygotowawczego, użyj:

```
1 git add <nazwa_pliku>
```

lub aby dodać wszystkie zmiany:

```
1 git add . (kropka jest bardzo ważna)
```

Charakterystyka:

- Umożliwia utworzenie zatwierdzenia zawierającego tylko kilka plików, nawet jeśli zmodyfikowano wiele z nich.
- To, co znajduje się w obszarze roboczym, zostanie dokładnie zapisane po uruchomieniu polecenia git commit.

Przykładowy schemat blokowy:

Modyfikujesz trzy pliki: index.php, style.css, README.md.

W kolejnym zatwierdzeniu wystarczy uwzględnić pliki index.php i README.md.

Wykonujesz:

```
1 git add index.php README.md
```

Plik style.css nie jest uwzględniany w obszarze roboczym, do późniejszego zatwierdzenia.

Magazyn

Repozytorium (lub wewnętrzna baza danych) to miejsce, w którym Git trwale przechowuje historię zmian.

Jego fizyczną reprezentacją jest folder .git w projekcie.

Kiedy biegasz:

```
1 git commit -m "Zatwierdź wiadomość"
```

Machine Translated by Google

Git pobiera dane z obszaru przygotowawczego i zapisuje je w repozytorium jako nową „migawkę” projektu (zatwierdzenie) z następującymi poleceniami:

Dokładna zawartość plików w tamtym czasie.

- Autor i jego adres e-mail.
- Data i godzina.
- Wiadomość, którą dodałeś.

Każde zatwierdzenie ma unikalny identyfikator (hash), który umożliwia:

- Wróć do tego momentu w czasie.
- Porównywanie zmian pomiędzy zatwierdzeniami.
- Utwórz gałęzie z tego stanu.

Podsumowanie przepływu podstawowego

Typowy przepływ pracy w systemie Git można podsumować w trzech krokach:

Praca w katalogu roboczym: Pliki tworzysz i modyfikujesz normalnie.

Przygotuj zmiany w obszarze przygotowawczym

```
1 git dodaj plik1 plik2
```

albo

```
1 git add .
```

Zapisz zmiany w repozytorium (historia):

```
1 git commit -m „Wyczyść opis zmian”
```

Ten przepływ będzie powtarzany przez cały czas trwania projektu.

Machine Translated by Google

MODUŁ 4: Zarządzanie plikami i zmianami

Cel

Opanuj podstawowe narzędzia umożliwiające kontrolowanie, przeglądanie i zarządzanie zmianami w plikach projektu: przeglądaj różnice, usuwaj i zmieniaj nazwy plików, a także wykluczaj zbędne elementy z kontroli wersji.

Wyświetl różnice (git diff)

Polecenie git diff pokazuje różnice między różnymi stanami plików: między katalogiem roboczym a obszarem przejściowym lub między zatwierdzeniami.

Główne zastosowania:

```
różnica git                    # Zmiany w katalogu roboczym i obszarze przejściowym
git diff --staged              # Zmiany w obszarze przygotowawczym w porównaniu z ostatnim zatwierdzeniem
git diff HEAD                  # Całkowita liczba zmian w porównaniu z ostatnim zatwierdzeniem
git diff commit1 commit2 # Różnice między dwoma konkretnymi zatwierdzeniami
```

Przykład praktyczny:

```
# Masz zmodyfikowany, ale nie dodany plik index.html
$ git diff
różnice --git a/index.html b/index.html
indeks e3f1e50..86b2f06 100644
--- a/index.html
+++ b/index.html
@@ -1,4 +1,4 @@
<h1>Mój projekt</h1>
- <p>Stara wersja</p>
+<p>Nowa i ulepszona wersja</p>
```

Linie oznaczone znakiem - oznaczają usuniętą treść,

+ dodana treść.

Modyfikuj pliki i śledź zmiany

Kompletny przepływ pracy umożliwiający śledzenie wszelkich modyfikacji:

Machine Translated by Google

Zobacz aktualny status:

```
status git
```

Sprawdź dokładnie, co się zmieniło:

```
różnica git
```

Przygotuj się na zmiany:

```
git dodaj archivo_modificado.html
```

Sprawdź co zostało przygotowane:

```
git diff --staged
```

Potwierdź zmiany:

```
git commit -m „Aktualizowanie głównej zawartości”
```

> Porada dla profesjonalistów: Zawsze używaj poleceń `git status` i `git diff` przed wykonaniem polecenia `git add`, aby mieć pewność, co dokładnie chcesz dodać.

Usuń pliki (git rm)

Aby bezpiecznie usunąć pliki z projektu i kontroli Git:

```
git rm nazwa_pliku.txt
git rm *.log                    # Usuń wszystkie pliki .log
git rm -r folder/               # Usuń cały folder
```

Ważne opcje:

`git rm --cached archivo.txt` Usuwa plik z kontroli Git, ale zachowuje plik fizycznie

Po uruchomieniu `git rm` musisz zatwierdzić usunięcie, aby je zapisać.

Przykład:

Machine Translated by Google

```
# Usuń przestarzały plik
$ git rm old_script.php
rm 'viejo_script.php'
$ git commit -m "Usuwam przestarzały skrypt"
```

Zmień nazwy plików (git mv)

Git posiada specjalne polecenie do zmiany nazw lub przenoszenia plików, które poprawnie rejestruje zmiany w historii:

```
git mv stary_plik.txt nowy_plik.txt
git mv script.js src/script.js                    # Służy również do poruszania się
```

Zalety systemu w porównaniu do mv:

- Prawidłowo rejestruje, że plik został zmieniony na nazwę lub przeniesiony
- Przechowuje historię zatwierdzania pliku
- Automatycznie aktualizuje obszar przygotowawczy

Przykład:

```
$ git mv index.html src/index.html
$ git status
Na odgałęzieniu głównym
zmiany do zatwierdzenia:
przemianowano:      index.html -> src/index.html
$ git commit -m "Przenieś indeks do folderu src"
```

Ignoruj pliki (.gitignore)

Plik .gitignore umożliwia wykluczenie plików i folderów z kontroli Git.

Należy bezwzględnie unikać śledzenia plików tymczasowych, konfiguracji lokalnych, kompilacji itp.

Utwórz .gitignore:

Machine Translated by Google

```
# W katalogu głównym repozytorium
dotknij .gitignore
```

Typowe przykłady dla programistów stron internetowych:

```
# Zależności
moduły_węzłowe/
dostawcy/

# Tymczasowe pliki systemowe
.DS_Store
Thumbs.db

# Dzienniki
*.log
npm-debug.log*

# Skompilowane kompilacje
dyst./
kompilacja/
*.min.js
*.min.css

# IDE i edytorów
.vscode/
.pomysł /
*.swp
*.swo
# Ustawienia lokalne
config.local.php
.env
wp-config.php
```

Machine Translated by Google

Ważne zasady:

Jedna linia = jedna zasada

Rozpocznij komentarz

*.log ignoruje wszystkie wpisy .log

**/node_modules/ ignoruje node_modules w dowolnym podfolderze

!important_file.log robi wyjątek (tak, postępuj zgodnie z tym plikiem)

Aby zastosować .gitignore do plików, które już zostały obserwowane:

```
1 git rm -r --cached .
2 git add .
3 git commit -m "Prawidłowo zastosowałem .gitignore"
```

Zalecany kompletny przepływ pracy

```
1 status git                                  # Widzę aktualny status
2 różnice git                                 # Przeglądam konkretne zmiany
3 git add archivo.html                        # Przygotowuję konkretne pliki
4 git diff --staged                           # Potwierdzam co zostanie zobowiązane
5 git commit -m "Wyczyść wiadomość" # Zapisz zmiany
```

Zawsze zachowuj tę kolejność, aby mieć pełną kontrolę nad tym, co uwzględniasz w każdym zatwierdzeniu.

Machine Translated by Google

MODUŁ 5: Gałęzie

Cel

Zrozumienie potencjału gałęzi Git umożliwiających pracę nad równoległymi wersjami projektu, tworzenie nowych funkcji, naprawianie błędów i scalanie zmian w kontrolowany sposób.

Czym jest oddział?

Gałąź w Gicie to niezależna linia rozwojowa umożliwiająca jednoczesną pracę nad różnymi wersjami tego samego projektu.

Każda gałąź reprezentuje lekki wskaźnik, który wskazuje na konkretne zatwierdzenie w historii.

Główne zalety:

- Opracowywanie nowych funkcji bez wpływu na wersję stabilną
- Napraw krytyczne błędy w produkcji bez wpływu na rozwój
- Eksperymentuj z pomysłami bez ryzyka.
- Współpracuj z zespołami równolegle

Prosta wizualizacja:

```
              D (main HEAD)
              ↑
A ← B ← C ──┼── E (nueva-funcionalidad HEAD)
              ↑
           Punto de divergencia
```

Oddział główny/główny

Domyślnie Git automatycznie tworzy gałąź główną o nazwie main (od wersji Git 2.28) lub master (od starszych wersji).

Ta gałąź zwykle reprezentuje:

- Stabilna wersja projektu
- Kod gotowy do produkcji
- Główny punkt odniesienia

Machine Translated by Google

Sprawdź aktualną gałąź:

```
gałąź git
* główne
```

Gwiazdka * Wskaż, nad którą gałęzią aktualnie pracujesz.

Utwórz gałęzie (git branch)

Aby utworzyć nową gałąź z bieżącego zatwierdzenia:

```
gałąź git nazwa-gałęzi
```

Przykład praktyczny:

```
# Jesteś w mainie, commit D
$ git branch nowa-funkcja
$ gałąź git
  * główny
    nowa funkcja
```

Nowa gałąź zostaje utworzona, ale nie przełączasz się na nią automatycznie.

Aby zobaczyć wszystkie oddziały (w tym te oddalone):

```
gałąź git -a
```

Twórz i zmieniaj jednocześnie:

```
git checkout -b nowa-funkcja
```

(lub w nowoczesny sposób: git switch -c new-function)

Przełączanie gałęzi (git checkout, git switch)

Metody zmiany gałęzi:

```
git checkout nazwa-gałęzi          # Metoda klasyczna
git switch branch-name             # Nowoczesna metoda (Git 2.23+)
```

Machine Translated by Google

Pełny przykład:

```
# 1. Utwórz gałąź z głównej
$ git checkout -b feature-login

# 2. Wykonujesz pracę w feature-login (zatwierdzenia E, F)
$ git commit -m „Dodaję system logowania”

# 3. Wróć do głównego
$ git checkout main

# 4. main jest nadal na D, feature-login jest na F
```

Efekt wizualny:

```
main:          A ← B ← C ← D
                           ↙
feature-login: A ← B ← C ← D ← E ← F
```

Ważna uwaga: Git pozwala na zmianę gałęzi tylko wtedy, gdy w katalogu roboczym nie ma niezatwierdzonych zmian.

Scalenie gałęzi (git merge)

Aby połączyć jedną gałąź z inną (zwykle rozwój główna):

```
gałąź docelowa git checkout
git merge branch-origin
```

Typowy przepływ:

```
# 1. Upewnij się, że main jest aktualny
$ git checkout main
$ git pull origin main

4
```

Machine Translated by Google

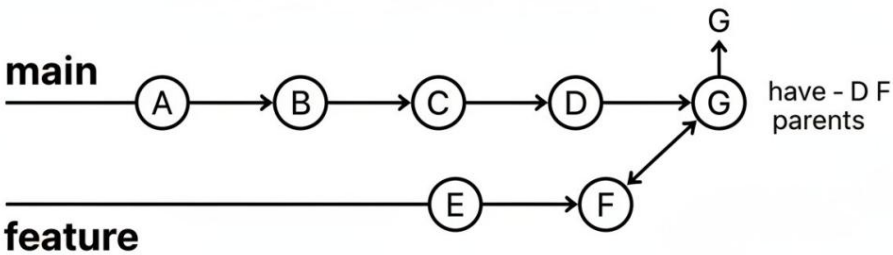
```
# 2. Przejdź do oddziału z nowymi zmianami
$ git checkout feature-login

#3. Połącz funkcję logowania z główną
$ git checkout main
$ git merge feature-login
```

Rodzaje scalania:

Szybkie przewijanie (automatyczne, bez konfliktów):

Zatwierdzenie scalenia (rozbieżność):



Konflikty związane z fuzjami

Konflikt występuje, gdy Git nie może automatycznie połączyć dwóch gałęzi, ponieważ zmodyfikowały te same wiersze tego samego pliku.

Git wyraźnie zaznacza miejsca występowania konfliktów:

```
<<<<<<< HEAD (główna)
<p>Wersja główna </p> =======
<p> Wersja
do logowania do funkcji </p> >>>>>>>
feature-login
```

Sekcje są ograniczone przez:

Machine Translated by Google

<<<<<<< HEAD zmiany gałęzi docelowej

======= separator

>>>>>>> gałąź źródłowa zmiany gałęzi źródłowej

Rozwiązywanie konfliktów

Kroki rozwiązywania konfliktów:

- Otwórz pliki powodujące konflikt w edytorze
- Edytuj ręcznie, wybierając wersję, którą chcesz zachować, lub łącząc obie
- Usuń znaczniki konfliktu (<<<<<<<, =======, >>>>>>>)

Dodaj rozwiązane pliki:

```
1 git add conflict-file.html
```

Zakończ scalanie:

```
1 zatwierdzenie git
```

(Git automatycznie otwiera edytor z domyślnym komunikatem)

Przykład rozwiązania:

```
1
2 <!-- PRZED (konfliktem) -->
3 <<<<<<< GŁOWA
4 <p>Witaj użytkowniku</p>
5 =======
6 <p>Witamy, Juan Pérez</p>
7 >>>>>>> funkcja-logowania
8
9 <!-- PO (rozwiązanym) -->
10 <p>Witamy, <span id="username">Juan Pérez</p>
11
```

Zalecany przepływ pracy z gałęziami

```
1 git checkout main # Stabilna podstawa
```

Machine Translated by Google

```
2 git pull                                    # Aktualizacja
3 git checkout -b feature/new-function # Nowa gałąź
4 # Praca, zatwierdzenie na gałęzi
5 git checkout main                           # Powrót do strony głównej
6 funkcja łączenia git /nowa funkcja          # Scal zmiany
7 git push origin main                        # Zmiana biegu na wyższy
```

Konwencja nazewnictwa:

```
1    cecha/nazwa-funkcji
3    poprawka błędu/błąd krytyczny
5    poprawka/wersja-1.2.1
7    wydanie/v1.0.0
```

Dzięki opanowaniu gałęzi masz pełną moc Gita, aby pracować nad wieloma wersjami jednocześnie.

Machine Translated by Google

MODUŁ 6: Zdalne repozytoria (GitHub / GitLab / Bitbucket)

Cel

Naucz się pracować z repozytoriami na platformach współpracy, synchronizować zmiany między komputerem lokalnym a chmurą i stosować standardowe przepływy pracy w ramach współpracy.

Czym jest zdalne repozytorium?

Zdalne repozytorium to kopia Twojego projektu umieszczona na serwerze w internecie (GitHub, GitLab, Bitbucket itp.).

Działa to tak:

- Automatyczne tworzenie kopii zapasowej kodu
- Punkt współpracy zespołowej
- Platforma do przeglądów kodu (żądania ściągnięcia)
- Integracja z CI/CD i zautomatyzowanymi wdrożeniami

Lokalnie: Twoja pełna kopia na Twoim komputerze

Zdalnie: kopia zapasowa w chmurze, dostępna z dowolnego miejsca

Załóż konto na GitHub (lub innej platformie)

GitHub (github.com) jest najpopularniejszą platformą:

Przejdź do github.com Zarejestruj się

Załóż bezpłatne konto, podając adres e-mail i hasło

Zweryfikuj swój adres e-mail

Skonfiguruj 2FA (uwierzytelnianie dwuskładnikowe) niezbędne

Alternatywy:

GitLab (gitlab.com) większe skupienie na DevOps

Bitbucket (bitbucket.org) natywna integracja z Jira

GitHub Enterprise dla firm

Machine Translated by Google

Klonowanie repozytoriów (git clone)

Klonowanie powoduje pobranie kompletnego zdalnego repozytorium na Twój komputer:

```
klon git https://github.com/usuario/repositorio.git
git clone git@github.com:user/repository.git # SSH (zalecane)
```

Co robi git clone:

- Pobierz CAŁĄ historię zatwierdzeń
- Utwórz folder o nazwie repozytorium
- Automatyczna konfiguracja zdalnego źródła
- Znajduje się na głównej gałęzi

Przykład:

```
$ git clone https://github.com/youruser/my-project.git
dolary CD mój-projekt
$ git remote -v
pochodzenie https://github.com/youruser/my-project.git (pobierz)
pochodzenie https://github.com/youruser/my-project.git (push)
```

Połącz lokalne repozytorium ze zdalnym repozytorium

Jeśli masz już lokalne repozytorium i chcesz je połączyć:

```
git zdalne dodawanie origin https://github.com/usuario/repositorio.git
```

Sprawdź połączenie:

```
git remote -v
gałąź git -vv                    # Pokaż bieżącą gałąź + zdalną
```

W przypadku wielu pilotów:

```
git zdalne dodanie upstream
https://github.com/organizacion/repositorio.git
polecenia git push
```

Machine Translated by Google

Prześlij swoje lokalne zatwierdzenia do zdalnego repozytorium:

```
1 git push origin main                    # Pierwszy raz
2 polecenia git push                      # Po (w skrócie)
3. Funkcja git push origin — nowość       # Konkretna gałąź
```

Pierwszy raz z nowym oddziałem:

```
1 git push -u origin main                 # -u skonfiguruj śledzenie upstream
```

Tylko wtedy:

```
1 polecenie git push
```

Sprawdź status zdalny:

```
1 status git
2 # Twoja gałąź jest aktualna z 'origin/main'.
3 git pull
```

Pobierz i scal zmiany z kopii zdalnej do kopii lokalnej:

```
1 git pull origin main
2 git pull (domyślnie)                    # Skróty (używa konfiguracji przez)
```

Do czego służy polecenie git pull?

```
1 git pull = git fetch + git merge
```

Typowe sytuacje:

```
1 # Aktualizacja główna
2 $ git checkout main
3 $ git pull
```

```
1 # Aktualizacja konkretnej gałęzi
2 $ funkcja git checkout-nowość
3 $ git pull origin feature-new
```

Machine Translated by Google

```
pobieranie git
```

Pobierz zmiany ze źródła zdalnego BEZ ich automatycznego scalania:

```
git pobierz pochodzenie
git fetch --all
```

Kluczowa różnica:

```
git fetch     Po prostu pobierz, ty decydujesz, co zrobić
git pull      Pobierz + automatyczne scalanie
```

Wyświetl zmiany zdalne bez scalania:

```
pobieranie git
git log HEAD..origin/main                    # Nowe zdalne zatwierdzenia
git log --oneline --graph origin/main
```

Podstawowy przepływ pracy w trybie współpracy (np. 3 programistów)

1. Klonuj początkowy projekt

```
klon git https://github.com/equipo/proyecto.git
```

2. Utwórz gałąź funkcji

```
    git checkout -b funkcja/rejestracja-użytkownik
```

3. Pracuj lokalnie

```
    # Dokonaj commitów
    git add .
    git commit -m „Dodaję rejestrację użytkownika”
```

4. Aktualizacja z głównej (dobra praktyka)

```
    git checkout główny
    git pull origin main
    funkcja git checkout /rejestracja-użytkownika
    git rebase main                    # Opcjonalnie: rebase w celu wyczyszczenia historii
```

Machine Translated by Google

5. Zmiana biegów

```
git push origin feature/user-registration
```

6. Utwórz żądanie ściągnięcia na GitHubie

```
# W interfejsie internetowym    Porównaj i wyślij żądanie ściągnięcia
```

7. Przegląd zespołu Połącz z głównym

8. Aktualizacja lokalna

```
git checkout główny
git pull origin main
```

Typowe codzienne polecenie

```
git checkout main && git pull && git checkout - && git rebase main
```

Szybkie polecenia referencyjne

Działanie	Rozkaz
Wyświetl piloty	git remote -v
Dodaj pilota	git zdalne dodawanie adresu URL pochodzenia
Klon	adres URL klonowania git
W górę (pierwszy raz)	git push -u origin main
Idź w górę (rutyna)	git push
Pobierz + scal	git pull
Po prostu pobierz	pobierz git
Wyświetl status synchronizacji git	

Dzięki opanowaniu obsługi zdalnych repozytoriów masz dostęp do kompletnego przepływu pracy, który pozwala Ci pracować indywidualnie lub w zespole.

Machine Translated by Google

MODUŁ 7: Praca zespołowa

Cel

Poznaj podstawowe narzędzia i praktyki umożliwiające efektywną pracę w zespołach korzystających z Gita: forki, przeglądy kodu i profesjonalne strategie rozgałęzień.
Widelce

Fork to osobista kopia repozytorium, którego nie jesteś właścicielem, przechowywana na Twoim koncie.
Używa się go głównie w projektach typu open source lub gdy pracujesz w organizacji, w której nie masz bezpośrednich uprawnień do zapisu.

Typowy przepływ widełkowy:

```
# Fork oryginalnego repozytorium     Twoje konto

git klonuje twój fork

git zdalne dodawanie repozytorium upstream -original

Utwórz gałąź     Praca     Prześlij do swojego forka

5. Żądanie ściągnięcia z forka     Oryginał
```

Przykład praktyczny:

```
# Sklonuj SWÓJ fork

klon git https://github.com/TU-USUARIO/proyecto-fork.git

# Połącz się z oryginałem (upstream)

git zdalne dodanie upstream https://github.com/ORIGINAL-AUTHOR/proyecto.git


# Zweryfikuj

git remote -v

# pochodzenie          https://github.com/TWOJA-NAZWA-UŻYTKOWNIKA/fork-project.git

# upstream https://github.com/ORIGINAL-AUTOR/proyecto.git
```

Żądania ściągnięcia / żądania scalenia

Pull Request (GitHub) i Merge Request (GitLab) to formalne prośby o scalenie zmian z jednej gałęzi/rozwidlenia do innej.

Machine Translated by Google

Główny cel:

- Przegląd kodu przez zespół
- Dyskusja na temat wdrożenia
- Testowanie automatyczne (CI/CD)
- Dokumentacja decyzyjna

Życie żądania ściągnięcia:

1. Deweloper tworzy PR z funkcji główny
2. Zespół przegląda kod, komentuje i sugeruje zmiany
3. Deweloper odpowiada/aktualizuje
4. CI/CD automatycznie uruchamia testy
5. Połącz główny

Przegląd kodu

Przegląd kodu to proces, w którym współpracownicy sprawdzają kod przed jego scaleniem:

Co sprawdzić:

- Funkcjonalność działa poprawnie
- Styl i standardy (linting)
- Obsługa błędów
- Wydajność
- Bezpieczeństwo
- Dokumentacja

Co powinieneś zrobić jako recenzent:

1. Przeczytaj cały PR (tytuł, opis)
2. Przeglądaj poszczególne zatwierdzenia (nie tylko różnice)
3. Przetestuj lokalnie, jeśli jest to skomplikowane
4. Komentuj konstruktywnie:
   - „Dlaczego taka implementacja?”
   - „Sugeruję użycie X zamiast Y”
   - "Doskonale, popieram"

Machine Translated by Google

5. Zatwierdź lub poproś o zmiany

Konwencja komentarzy:

Bloker (nie scalaj)

Ważna sugestia

Komentarz/wyjaśnienie

Zatwierdzony

Dobre praktyki w zespołach

Wyczyść komunikaty zatwierdzenia:

Zły

```
1 git commit -m "fix"
```

Dobrze

```
1 git commit -m „Naprawiono: sprawdź poprawność adresu e-mail przed zapisaniem użytkownika”
```

Konwencjonalne zatwierdzenia

wyczyn: dodanie uwierzytelniania OAuth2

poprawka: poprawna walidacja formularza

dokumenty: zaktualizuj plik README o nowe instrukcje

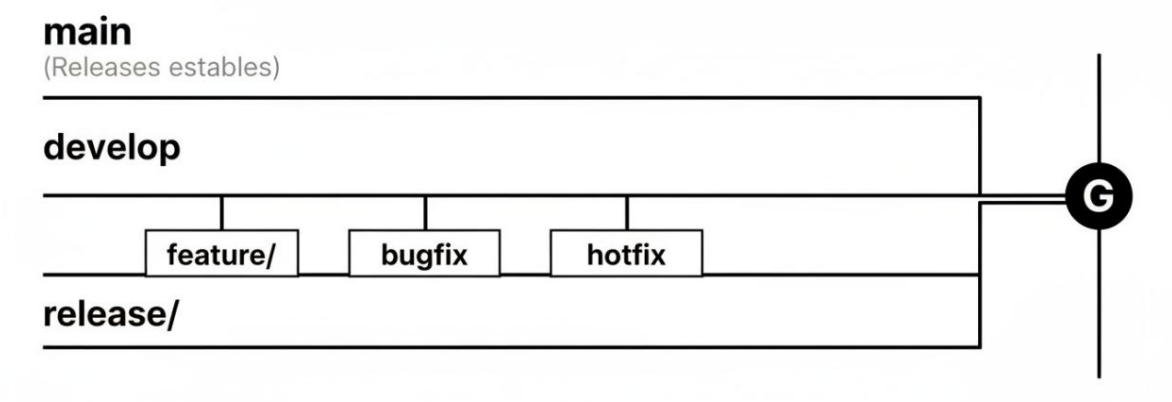
Złote zasady:

1. Małe i atomowe zatwierdzenia (1 zmiana = 1 zatwierdzenie)
2. Główna gałąź ZAWSZE stabilna (nigdy nie zatwierdzaj bezpośrednio)
3. Pobieranie przed przesyłaniem (git pull origin main)
4. Używaj gałęzi z prefiksami: feature/, bugfix/, hotfix/
5. Małe PR-y (preferowane <400 linii)
6. Nie przepisuj współdzielonej historii (nie zmieniaj bazy w pliku głównym)

Strategie rozgałęzień (Git Flow – wprowadzenie)

Git Flow to popularna strategia dla zespołów:

Machine Translated by Google



Główne gałęzie:

Oddział	Zamiar	Ochrona
główny	Stabilna produkcja	Maksymalny
rozwijać	Ciągła integracja	Wysoki
funkcja/*	Nowe funkcje	Niski
uwolnienie/*	Przygotuj wersje	Przeciętny
poprawka/*	Krytyczne błędy w produkcji	Wysoki

Podstawowy przepływ pracy z Gitem:

```
1 git checkout develop

2 git checkout -b feature/registration-user

3 # Rozwijaj     zatwierdzaj     wypychaj

4 git checkout develop

5 funkcja łączenia git /rejestracja użytkownika

6 git push origin develop

7 Utwórz PR rozwijaj     główny, gdy gotowy
```

Polecenie uruchomienia Git Flow:

```
1 git flow init
```

Machine Translated by Google

Kompletny wspólny przepływ pracy (podsumowanie)

Wywoływacz

```
1 git checkout main && git pull
2 git checkout -b feature/task-name
3 # Praca + zobowiązania
4 git push -u origin feature/task-name
5. Utwórz PR na GitHub/GitLab
```

Dyrygent

```
1 git pobierz pochodzenie
2 git checkout -b review/feature/task-name origin/feature/ task -name
3 Test     Komentarz do PR
```

Po scaleniu

```
1. git checkout main i git pull origin main
```

Dzięki tym praktykom Twój zespół będzie w pełni profesjonalny w zakresie Git: sprawdzony kod, czysta historia i bezpieczne wdrożenia.

Machine Translated by Google

MODUŁ 8: Cofanie zmian

Cel

Odzyskaj pełną kontrolę nad każdym błędem, nie tracąc przy tym efektów pracy: od niezapisanych zmian po cofanie udostępnionych zatwierdzeń — dzięki odpowiedniemu narzędziu na każdą sytuację.
sytuacja.

Cofnij niepotwierdzone zmiany.

Zmiany w katalogu roboczym (zmodyfikowane, ale bez git add):

```
1 git checkout -- plik.txt                    # Przywraca plik do ostatniego zatwierdzenia

2 git restore archivo.txt                     # Git 2.23+ (czytelniejszy)
```

Pliki w obszarze przygotowawczym (już z dodanym poleceniem git, ale bez zatwierdzenia):

```
1 git restore --staged archivo.txt # Usuwa z przechowalni, zachowuje zmiany

2 git reset HEAD file.txt                     # Metoda klasyczna
```

Przykład praktyczny:

```
 1 # Stan początkowy

 2 $ git status

 3 zmodyfikowano: index.html

 4

 5 # Cofnij WSZYSTKIE niepotwierdzone zmiany

 6 $ git restore .

 7 # lub

 8 $ git checkout --

 9

10 # Tylko określone pliki

11 $ git restore index.html style.css
```

Machine Translated by Google

git checkout --

Przywraca określone pliki z katalogu roboczego do stanu ostatniego zatwierdzenia:

```
1 git checkout -- plik.txt
```

Działa dla:

- Zmodyfikowane pliki (katalog roboczy)
- Pliki przygotowane (również usuwane z przygotowania)

Przykład:

```
1 $ git status
2 zmiany nie zostały przygotowane do zatwierdzenia:
3         zmodyfikowano: src/app.js
4
5 $ git checkout -- src/app.js
6 $ git status
7 nic do zobowiązania
```

przywróć git

Nowoczesne polecenie (Git 2.23+), które zastępuje polecenie git checkout i polecenie git reset HEAD:

Działanie	Rozkaz
Cofnij zmiany Działa Informator	git restore file.txt
Usuń z obszaru przygotowawczego	git restore --staged file.txt
Obie akcje	git restore --staged --worktree plik.txt
Wszystko na raz	git restore .

Zaleta: Znacznie bardziej przejrzysta składnia niż w starych poleceniach.

Machine Translated by Google

git reset (miękki, mieszany, twardy)

`git reset` przesuwa wskaźnik HEAD do tyłu w historii, w 3 trybach:

Tryb	Obszar roboczy katalogu tymczasowego		Zobowiązania
miękki	Zachowaj zmiany	Utrzymaj inscenizację	Przesuń GŁOWĘ
mieszane (domyślne)	Zachowaj zmiany	Czyszczenie sceny	Przesuń GŁOWĘ
twardy	Usuń wszystko	Czyszczenie sceny	Przesuń GŁOWĘ

Przykład praktyczny:

```
# Masz commity ABCD (HEAD w D)

# Powrót do C, pozostawiając zmiany z D gotowe do nowego zatwierdzenia
$ git reset --soft HEAD~1

# Powrót do C, zachowując zmiany z D bez etapu
$ git reset HEAD~1                    # domyślnie mieszane

# Powrót do C, całkowite usunięcie zmian z D
$ git reset --hard HEAD~1
```

> UWAGA: polecenie git reset --hard spowoduje nieodwracalne usunięcie Twojej pracy, jeśli nie posiadasz kopii zapasowej.

przywróć git

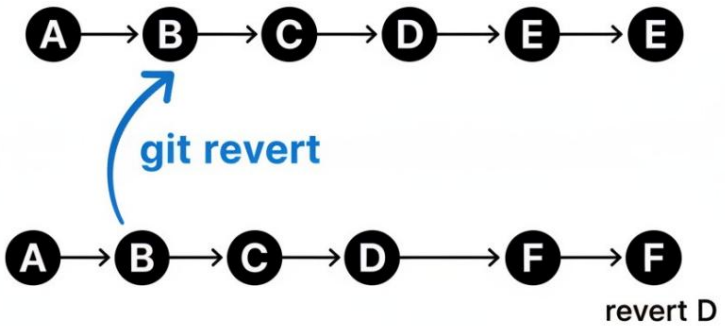
Utwórz NOWE zatwierdzenie, które cofa zmiany z poprzedniego zatwierdzenia.

BEZPIECZNY dla współdzielonej historii (nigdy nie nadpisuje zatwierdzonych już zmian).

```
git revert <commit-hash>
```

Machine Translated by Google

Przykład:




```
1 # Cofnij zatwierdzenie D (hash: abc123)

2 $ git revert abc123

3 # Git otwiera edytor dla komunikatu: „Przywróć „Wiadomość z D”"

4 $ git push
```

Zalety:

- Nie przepisuje historii
- Bezpieczeństwo dla urządzeń (bezpośrednie przesyłanie git)
- Udokumentuj, co zostało odwrócone i dlaczego

Kiedy używać każdego z nich

Sytuacja	Zalecane polecenie
Niezapisane zmiany (katalog roboczy)	git restore file.txt
Pliki przygotowane przez pomyłkę	git restore --staged file.txt
Ostatnie lokalne zatwierdzenie, NIE wypchnięte	git reset --soft HEAD~1
Współdzielone zatwierdzenie (już zdalne)	git revert <hash>
Całkowita katastrofa, zaczynanie od zera	git reset --hard HEAD~1
Pobierz konkretny plik	git checkout -- plik.txt

Machine Translated by Google

Stół bezpieczeństwa

Rozkaz	Ubezpieczenie sprzętu	Odzyskiwalny
przywróć git	Zawsze	
git reset --soft	Tylko lokalnie	
git reset --hard	Nigdy nie udostępniane	
przywróć git	Zawsze	

Przykład kompletnego przepływu odzyskiwania

```
# BŁĄD: Zmodyfikowałem 5 plików, 3 przez pomyłkę ustawiłem jako etap przygotowawczy, zatwierdziłem 2
$ git status
zmiany wprowadzone: plik1.js plik2.js
Zmiany nieuwzględnione : file3.css file4.html file5.txt

# 1. Usuń nieprawidłowe ustawienie
$ git restore --staged plik1.js plik2.js

# 2. Cofnij niechciane zmiany
$ git restore file3.css file5.txt

# 3. Chcę zachować tylko plik file4.html
$ git add archivo4.html
$ git commit -m "Tylko przydatne zmiany"
```

Złota zasada: zawsze używaj `git status` przed każdym poleceniem destrukcyjnym.

Gdy opanujesz te polecenia, nigdy więcej nie stracisz swojej pracy z powodu błędów.

Machine Translated by Google

MODUŁ 9: Zaawansowany Git (opcjonalnie)

Cel

Opanuj zaawansowane polecenia dotyczące profesjonalnych przepływów pracy: zapisz pracę tymczasową, oczyść historię, wybieraj konkretne zatwierdzenia i automatyzuj powtarzające się zadania.

git stash

Zapisz zmiany bez konieczności tymczasowego przełączania gałęzi lub czyszczenia gałęzi roboczej Informator.

```
1 git stash push -m "Formularz logowania WIP"    # Zapisz z wiadomością
2 lista git stash                                # Wyświetl skrytki
3 git stash pop                                  # Przywróć ostatnie i usuń
4 git stash apply                                # Przywróć ostatni BEZ usuwania
5 git stash drop stash@{0}                       # Usuń konkretne
```

Typowy przepływ:

```
1 # 1. Masz niezatwierdzone zmiany, ale musisz zmienić gałąź
2 $ git status
3 zmiany nie zostały wprowadzone: index.html style.css
4
5 # 2. Zapisz pracę
6 $ git stash push -m "Częściowe zmiany logowania"
7
8 # 3. Zmień gałąź czystą
9 $ git checkout main
10 dolarów za git pull
11 # 4. Wróć do swojego oddziału i przywróć
13 $ git checkout feature-login
14 $ git stash pop
```

Porada: Używaj skrytek, aby szybko debugować, nie tracąc przy tym bieżącej pracy.

Machine Translated by Google

Rebase (git rebase)

Rebase nadpisuje historię, przenosząc zatwierdzenia na inną bazę (zwykle zaktualizowaną bazę główną).

```
git rebase main                    # Moje commity na mainie
git rebase -i HEAD~3               # Interaktywne przebazowanie (ostatnie 3)
```

Zanim:

tekst
główny: A B C

cecha: D E

Po:

tekst
główny: A B C

funkcja: D' E'

Zalecany przepływ:

```
funkcja git checkout-nowość
git rebase main                    # Wyczyść historię liniową
git push --force-with-lease        #        wspólna opieka
```

Zasada: Nigdy nie zmieniaj bazy zatwierdzeń już przesłanych do współdzielonego serwera zdalnego.

Wybiórczo

Kopiuj konkretne zatwierdzenia z jednej gałęzi do drugiej, bez scalania wszystkiego.

```
git cherry-pick <zatwierdzenie hash>
git cherry-pick commit1 commit2    # Wiele zatwierdzeń

typowe scenariusze:

# 1. Poprawka w głównej wersji, którą chcesz również wdrożyć
$ git checkout develop
```

Machine Translated by Google

```
$ git cherry-pick abc1234d # Kopiuj konkretną poprawkę

# 2. Przydatna funkcja, której potrzebujesz TERAZ w wersji głównej
$ git cherry-pick 1a2b3c4d
```

Zaleta: Chirurgiczna precyzja — tylko takie zaangażowanie, jakiego potrzebujesz.

Tagi i wersje

Tagi oznaczają konkretne punkty w historii (wersje, wydania).

```
tag git v1.0.0                                  # Tag lekki
git tag -a v1.0.0 -m "Release 1.0" # Tag z adnotacjami (zalecane)
tagi git                                        # Lista tagów
git checkout v1.0.0                             # Przejdź do konkretnego tagu
git push origin v1.0.0                          # Prześlij tag
git push --tags                                 # Prześlij wszystko
```

Konwencja SemVer:

```
wersja 1.0.0      # Początkowo stabilny
wersja 1.0.1      # Drobna poprawka
wersja 1.1.0      # Nowa funkcja
wersja 2.0.0      # Zmiany powodujące przerwanie działania
```

Git Hooks (Wprowadzenie)

Haki to automatyczne skrypty uruchamiane w odpowiedzi na określone zdarzenia (przed zatwierdzeniem, po scaleniu itd.).

Lokalizacja: .git/hooks/

Przydatne przykłady:

.git/hooks/pre-commit

```
#!/bin/sh
# Uruchom testy przed każdym zatwierdzeniem
testy npm
```

Machine Translated by Google

```
.git/hooks/pre-push
#!/bin/sh
# Sprawdź, czy main ma testy przed przesłaniem
jeśli [ "$(git rev-parse --abbrev-ref HEAD)" = "main" ]; wtedy
        test npm
fi
```

Utwórz plik wykonywalny:

```
chmod +x .git/hooks/pre-commit
```

Alias w Git

Przyspiesz długie polecenia za pomocą niestandardowych skrótów:

```
# Konfiguracja globalna
git config --global alias.st stash
git config --global alias.co checkout
git config --global alias.br gałąź
git config --global alias.ci commit
git config --global alias.stash-pop "stash pop"
git config --global alias.last "log -1 HEAD"
```

Następnie możesz użyć:

```
git st                  # git stash
git co main             # git checkout main
git br -a               # git branch -a
git ostatni             # git log -1 HEAD
```

Przydatne zaawansowane aliasy:

```
git config --global alias.unstage "restore --staged"
git config --global alias.up "pull --rebase"
git config --global alias.graph "log --oneline --graph --decorate -10"
```

Machine Translated by Google

Działanie	Szybkie polecenie	Sugerowany alias
Zapisz tymczasowo	git stash -m "WIP"	git st
Wyczyść historię	git rebase main	wstać
Kopiuj zatwierdzenie	git cherry-pick abc123	-
Ostatnie zatwierdzenie	git ostatni	git ostatni
Status graficzny	wykres git	wykres git

Codzienny profesjonalny przepływ pracy

```
# Jutro: synchronizacja
git co main && git up && git co -

# Późno: Szybkie zapisywanie pracy
git st -m "WIP przed spotkaniem"

# Koniec dnia: wyczyść historię
git rebase -i HEAD~3
git push --force-with-lease
```

Dzięki opanowaniu Advanced Git będziesz mieć pełną kontrolę nad każdą sytuacją zawodową.